

实验报告 — QEA 这一周:从重构到「让 agent 产 Excel」

日期 2026-06-16 · 仓库 evolving-quant-agent · 模型:worker+evolve = deepseek-v4-pro ,judge = qwen3.7-max (经 OpenRouter)

一句话总览

QEA 是「让一个 agent 的外壳(harness)自己进化」的实验框架。这一周:

- 1. 重构了打分和诊断机制 → 重构后第一次真跑,第一次有改动被真正采纳(修好了上阶段的老 bug)。
- 2. 但发现 agent 只会改「输出格式」、跳不出来 → 诊断根因:它物理上产不出 rubric 要的 Excel 文件。
- 3. 于是给 worker 加了产 Excel 的能力(v0.1)。

结果:Excel 真能产出来了,但光「能产文件」还不够 —— 会计类任务分数没涨,因为现在的打分器只看「有没有文件」,看不懂「内容对不对」。下一步的关键是做一个能读懂文件内容的打分器。

名词速查（看不懂时回来查这里）

词	一句话
harness / 外壳	agent 的脚手架,7 个槽:tool / middleware / skill / prompt / validator / memory / router。每轮只改一个。
进化循环	改一处 → 打分 → 比之前好就留下,否则撤回。
worker	真正干活、产出交付物的 agent。
evolve agent / proposer	决定「这轮改哪个组件、怎么改」的 meta agent。
B-pile 任务	~30 个真实 GDPval 金融/会计任务,要交一份 deliverable。
grader	LLM 按打分细则(rubric)逐条评,给一个 0-1 的百分制分。
format basin (格式泥潭)	proposer 反复只会改「输出格式」、跳不出来的退化状态。

1. 这一周的故事（一张图）

上阶段的老 bug: 诊断喂错了信息 → proposer 在"修幻觉出来的失败"(旧实验 0/7 被采纳)

▼ 这周第一件事

【重构】打分(grader/evaluator 拆开)+ 诊断(B-pile debugger,带防火墙)+ 简化

▼ 实验一: 重构跑通了吗?

【loop_test】✅ 有改动被采纳了(3/10,旧 0/7)→ bug 修好

⚠️ 但发现: 10 个改动全在改"格式" → 卡进 format basin

▼ 诊断

【根因】QEA 进化的只是"提示词文字", worker 产不出 Excel 文件 → 只能改格式 → 出不来

▼ 实验二: 给它加产 Excel 的能力(v0.1)

【xlsx_test】✅ 真产出 17 个 Excel; seed 分 +0.05

❌ 但会计尾部分没动、还是只改格式 → "能产文件"不等于"解决问题"

▼ 下一步

【关键路径】做一个看得懂文件内容的 grader(sub-project 3)

2. 背景: 上阶段卡在哪

- QEA 的终极目标是让 agent 外壳自己进化: 改一处 → 打分 → 好就留下。
- 上阶段的问题: 诊断模块给 proposer 喂了错误的失败信息(把代码任务的语义喂给了写作任务), proposer 于是在「修根本不存在的失败」, 旧实验 0/7 个改动被采纳, 完全在原地打转。
- 所以这周第一件事就是重构: 把打分和诊断重做, 并加一道防火墙(让 proposer 只看到「该改哪个组件」, 看不到答案本身, 防止它抄答案或修幻觉失败)。

3. 实验一(loop_test): 重构跑通了吗?

结论: ✅ 修好了, 但 ⚠️ 暴露了新瓶颈。

- 好的一面: 重构后第一次真跑(10 轮), 3 个改动被采纳(旧实验是 0/7)。说明 proposer 终于在做有用的事了。
- 意外的一面: 这 10 个改动全是「改格式」(加格式校验、加格式说明、加 schema...), 一个实质性的改动都没有。agent 卡进了格式泥潭。
- 分数从 0.609 涨到 0.662, 但会计/审计这一类一直最低, 只有 0.28。

10 个改动一览(全部围绕"格式"):

轮次	改动	结果
1-10	output_format_validator / format_instructions / output_schema / autoformatter / answer_formatter / format_compliance / format_reminder / format_schema / format_enforcer / output_sanitizer	只有 3 个被采纳,其余撤回

4. 诊断:为什么 agent 跳不出格式泥潭

结论:因为 QEA 进化的是「文字」,不是「代码」;而会计任务要的是一个 Excel 文件,文字永远变不出文件。

- agent 能改的所有组件,本质都是一段提示词文字,拼进 worker 的 prompt。
- worker 的「执行」(怎么跑代码、怎么产文件)是写死的框架代码,agent 碰不到。
- 会计任务的 rubric 要求「交一个 Excel 工作簿」。worker 只会写文字 → 这条永远不满足 → agent 看到「缺文件」这个失败,它唯一能做的就是改格式文字(但没用)→ 死循环。

和 AHE 的区别(AHE = QEA 的来源,一个完整版 agent 框架):

	QEA(本仓库)	AHE(原版)
进化的是	提示词文字	真代码 / 文件
worker	一次 LLM 调用 → 文字	真 agent:能跑代码、产文件
工具	只是文字描述(不真执行)	真能执行

一句话:QEA 是 AHE 的极简版——干净、便宜、好归因,但进化的是个「玩具外壳」。这也是为什么「产 Excel」这种能力必须由框架直接给(seed),而不能指望 agent 自己进化出来。

5. 实验二(xlsx_test):给 worker 加产 Excel 的能力(v0.1)

做了什么:让 worker 在需要时写一段 openpyxl 代码 → 框架在沙箱里跑它、产出 .xlsx → 再把文件渲染成文字、附到交付物里,让现有 grader 能看到。(16 次提交,6 个 TDD 任务,40 个测试全过。)

结论:✅ 机制成立,但 ❌ 光能产文件还不够。

	结果
✅ 真能产文件	全程产出 17 个真 Excel / 8 个任务(会计摊销表、相关性矩阵、Roth 转换表、payroll、P&L...)
✅ seed 分提升	0.609 → 0.660(+0.05,因为"有没有文件"这类条目现在能得分了)
❌ 但没解决问题	进化在上面没涨(0.660→0.659);会计尾部还是 0.29;3 个改动又全是格式型

为什么「能产文件」还不够:现在的 grader 是把 Excel 渲染成文字喂给 LLM 评。它能判「有没有文件、有没有这些 sheet」(结构)——所以 seed 分涨了 +0.05;但它判不了「内容对不对」(抽样逻辑、摊销是否真对平、数字准不准)。而会计任务真正要的是内容正确。
→ 所以 v0.1 做完了「产文件」这一半,剩下一半是「做一个看得懂内容的 grader」。

案例(让结论看得见):一个 GDPval 会计任务,worker 真的产出了一份多 sheet 的摊销表,而且能对平:

[文件: Aurisic_Prepaid_Schedule.xlsx]

Monthly Activity Summary					
月份	期初余额	新增	摊销	期末余额	
一月	0	622,721.83	103,786.97	518,934.86	← 0 + 622721.83 - 103786.97 = 518934.86 ✓
二月	518,934.86	13,830.29	106,092.02	426,673.13	

文件是真的、能对平的。但这个任务终态分仍只有 0.29 —— 「结构」能得分了,「内容是否全对」现在的 grader 判不了。这正是实验二的关键发现。

6. 学到的 / 待讨论

1. 进化的是文字、不是代码(§4)——这是 format basin 的根因,也决定了「能力靠 seed、流程靠进化」。
2. 「能产文件」≠「解决问题」(§5)——下一步必须做看得懂文件内容的 grader,否则文件这一维拿不到真信号。这把它从「可选」升为「关键路径」。
3. 打分噪声问题:这次的噪声地板低到 0.010,但跨「断点续跑」重新打分会抖动 ~0.017 → 那唯一被采纳的改动其实落在噪声里、不太可信。要让 worker 也多采几次样来压噪声。
4. proposer 的「观测」仍被格式主导:两次实验加起来,13 个改动全是格式型。说明诊断给它看的東西还是偏格式,要让它看到「内容缺陷」才会提实质改动。
5. 算力 provider:本想停 OpenRouter、改用 DashScope 的网关,但给的 token 被拒(invalid api-key),已回退 OpenRouter 跑完;新的 AnthropicLLM 接口已就位,换家只改 .env 。

7. 下一步(按优先级)

1. 【关键】做「看得懂内容」的 grader:Excel → 渲染成图 → 多模态 judge;会计这类先写逐格 (cell-level)确定性校验。让「内容对不对」能真打分。
2. 修打分噪声:worker 多采样 / 续跑用稳态分,让「被采纳」可信。
3. 让诊断 surface 内容缺陷:改 B-pile debugger,使 proposer 能提实质改动,而不是第 14 个格式改动。
4. 拿人类 gold 文件:把 GDPval 的人工标准答案文件下载缓存,为「对比 gold 评分」铺路。
5. 启动双 benchmark 路线(见 §8):先申请 FAB v2 的 license。

8. 更大的计划:要打的两个 benchmark

这是一次单独讨论的纪要,折进来作为中期路线。数字已 web 复核(✓);标「待复核」的没核实。

8.1 目标

让 QEA 在金融领域,从一个(近)无工具的 agent,进化成能同时应付文件输出和文本问答两类任务的金融 agent。

8.2 两个 benchmark 长什么样

	GDPval(金融子集)	FAB v2(Vals AI)
交付物	真实文件(Excel/Word/PPT/PDF)✓	纯文本答案 + 数字,无文件 ✓
怎么打分	专家盲比模型 vs 人类(主观、慢、贵)✓	程序化可硬验证(dealbreaker 门控 + 部分给分)✓
工具	官方不开源、无自动 grader	固定六工具,已开源 ✓
规模	1320 全集 / 220 开源,金融子集 25 题 ✓	927 题:公开 27 / 验证集 450(可申请 license)/ 测试 450 ✓
能进化循环吗	不能(主观打分,非硬验证)	能(但要解决样本量)
顶格成绩	顶级模型多数仍打不过人类(<50%)	部分给分 ~58%;最难的 Financial Modeling 只有 23% ✓

8.3 几条想清楚了的判断

- 一个进循环、一个做最终测试:FAB v2(可硬验证)当进化的打分信号;GDPval 当最后的「泛化测试」,只评一次。
- 能力靠 seed、不靠进化:头 80% 的提升来自现成脚手架(代码执行、检索、计算器),不值得让进化重新发明 → 用一个像样的 baseline 当进化起点。
- 会计审计任务是「文件输出 + 可硬验证」的桥梁:它的抽样公式、方差列、标记规则都能写成逐格校验,不靠主观打分 → 让进化在文件输出上拿到真梯度(正好接上 §7 第 1 项)。
- 成功判据:不追绝对榜首,改成「进化出的外壳 vs 强手工外壳的相对提升」,并按九类分别看。

8.4 还要定的 / 已经定的

待定

1. 进化的打分数据用哪个?倾向申请 FAB 的 450 验证集 license,待组内确认。(最高优先级,定了才好往下做)
2. 进化起点是「真·无工具」还是「baseline 外壳」?
3. 强手工 baseline 外壳具体怎么配?

已定

- GDPval 接入方式 = 当最终泛化测试 + 给会计类任务写逐格校验。
- 覆盖范围 = 先覆盖全九类、做尽量通用型,但分类报提升,Financial Modeling 作为最亮的子结论。

8.5 出处 (已复核)

- GDPval ✓:论文 [arXiv:2510.04374](#);OpenAI 官方页。
- FAB v2 ✓:Vals AI 页;论文 [arXiv:2508.00828](#);GitHub。
- 待复核:GDPval 文件交付占比的精确百分比、顶格 win-rate 区间;FAB「每题 2 小时」时限。